

**HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND
EDUCATION
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING
DEPARTMENT OF COMPUTER AND COMMUNICATION
ENGINEERING**



FINAL COURSE PROJECT

**SMART WATER TANK: LEVEL AND
FLOW MONITORING WITH
AUTOMATIC PUMP CONTROL**

MAJOR IN COMPUTER ENGINEERING

Students: [FULL NAME OF MEMBER 1]
Student ID: [ID]
[FULL NAME OF MEMBER 2]
Student ID: [ID]
[FULL NAME OF MEMBER 3]
Student ID: [ID]
[FULL NAME OF MEMBER 4]
Student ID: [ID]
[FULL NAME OF MEMBER 5]
Student ID: [ID]

Supervisor: Dr. HUYNH THE THIEN

HO CHI MINH CITY, 09/2026

**HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND
EDUCATION
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING
DEPARTMENT OF COMPUTER AND COMMUNICATION
ENGINEERING**



**FINAL COURSE PROJECT
SMART WATER TANK: LEVEL AND
FLOW MONITORING WITH
AUTOMATIC PUMP CONTROL
MAJOR IN COMPUTER ENGINEERING**

Students: [FULL NAME OF MEMBER 1]
Student ID: [ID]
[FULL NAME OF MEMBER 2]
Student ID: [ID]
[FULL NAME OF MEMBER 3]
Student ID: [ID]
[FULL NAME OF MEMBER 4]
Student ID: [ID]
[FULL NAME OF MEMBER 5]
Student ID: [ID]

Supervisor: Dr. HUYNH THE THIEN

HO CHI MINH CITY, 09/2026

ACKNOWLEDGMENTS

Throughout this final project for the course IoT Foundations and Applications, our group received careful guidance and valuable technical feedback from Dr. Huynh The Thien. His lectures on layered architecture, application layer messaging and time series data infrastructure provided the theoretical basis for most of the design decisions in the system we built.

We also thank the Department of Computer and Communication Engineering for granting us laboratory access, which allowed us to complete the hardware prototype and carry out the measurements reported here.

We would like to express our sincere and deepest gratitude.

Project group
(Signature and full name)

DECLARATION OF AUTHORSHIP

We declare that the entire content presented in this report is the result of our own research and implementation. The firmware, the backend service and the monitoring interface were written by us. The experimental data were collected directly on the physical prototype that we assembled. Every item taken from the course lectures, technical standards and scientific literature is cited at the point of use and listed in the references.

Project group
(Signature and full name)

ABSTRACT

This project builds a closed loop water management system able to measure water level and flow rate, drive a pump automatically, estimate consumed volume and detect abnormal hydraulic operating states. The system is implemented across the full layered model of the Internet of Things, from the perception layer on an ESP32 microcontroller, through a transport layer based on MQTT, a processing layer consisting of an ingestion service and a time series store, up to an application layer in the form of a real time monitoring dashboard.

The central design decision is to place the entire control law and the entire fault detection logic inside the edge microcontroller rather than on the server. As a result, level regulation and every safety constraint keep working while the network link is down. The controller is built as a finite state machine combining a hysteresis band with minimum on and off times, which removes the rapid switching behaviour that shortens pump life.

The evaluation covers calibration of the two main sensors against physical reference instruments, measurement of the closed loop response, a quantitative comparison between a single threshold policy and the proposed hysteresis policy, end to end latency measurement, and verification of system behaviour during a network outage.

Contents

1	INTRODUCTION	1
1.1	Motivation and problem statement	1
1.2	Scope and working method	1
2	THEORETICAL BACKGROUND AND SYSTEM REQUIREMENTS	3
2.1	Mandatory requirements of the assignment	4
2.2	Layered architecture and computation placement	5
2.3	Sensing, digitisation and actuation	5
2.4	Application layer messaging	6
2.5	Data infrastructure and the feedback sync principle	6
2.6	On off control with hysteresis	7
3	ARCHITECTURE AND HARDWARE DESIGN	8
3.1	Overall architecture	8
3.2	Function placement and edge autonomy	9
3.3	Sensor selection and signal conditioning	9
3.4	Power stage, supply and mechanical build	10
4	PROTOCOL AND COMMUNICATION	12
4.1	Design principles and topic tree	12
4.2	Message structure and fault codes	13
4.3	Command and acknowledgement mechanism	13
4.4	Connection lifecycle and availability monitoring	14
5	BACKEND, DATA MANAGEMENT AND INTERFACE	15
5.1	Backend architecture	15
5.2	Data schema and API	16
5.3	Monitoring interface	17
5.4	Device simulator	18
6	CONTROL AND SYSTEM INTELLIGENCE	19
6.1	Overview	19
6.2	Computed quantities	19
6.3	State machine and hysteresis	20

6.4	Fault detection rules	21
6.5	Manual mode and safety interlocks	22
6.6	Leak detection	22
7	SECURITY AND RELIABILITY	24
7.1	Threat model and mitigations	24
7.2	Authentication, authorisation and encryption	24
7.3	Reliability during a network outage	25
8	EXPERIMENTS, RESULTS AND CONCLUSION	27
8.1	Principles and reference instruments	27
8.2	Sensor calibration	28
8.3	Control response and fault detection	28
8.4	Latency and behaviour during an outage	29
8.5	Reducing transport latency at the radio	32
8.6	Limitations	33
8.7	Faults found on the bench	34
8.7.1	An inverted relay made the firmware believe the pump was off	34
8.7.2	An automatic direction level shifter consumed a third of the echoes	35
8.7.3	Fifty hertz on a floating signal line reads as plausible flow	35
8.7.4	A silently truncated message stopped the database without stopping anything else	36
8.7.5	A command that worked but was never confirmed	36
8.7.6	A robust filter gated by a fragile statistic	36
8.7.7	The level sensor is excellent, and unusable, in the same installation	37
8.8	Conclusion	37
8.9	Member contributions	38

List of Tables

2.1.1 Mapping of the mandatory requirements onto the report.	4
3.3.1 Main bill of materials.	9
4.1.1 MQTT topic tree and the reasoning behind each setting.	12
5.2.1 Main API endpoints.	16
6.3.1 Controller state set.	20
6.4.1 Fault detection rules and trigger conditions.	21
7.1.1 Threat scenarios and mitigations.	24
7.3.1 Four independent layers of overflow protection.	26
8.1.1 Plan of the eight experiments.	27
8.2.1 Calibration results for the two main sensors.	28
8.3.1 Control response and fault detection results.	29
8.4.1 Latency, outage behaviour and control policy comparison results.	31
8.5.1 Command round trip before and after disabling Wi-Fi power saving. Same device, same access point, same procedure.	32
8.9.1 Distribution of work among group members.	38

List of Figures

3.1.1 Architecture of the smart water tank system.	8
5.3.1 Monitoring dashboard in automatic mode.	17

List of Abbreviations

The following abbreviations are used throughout this report.

Abbreviation	Definition
IoT	Internet of Things
MQTT	Message Queuing Telemetry Transport
QoS	Quality of Service
LWT	Last Will and Testament
ACL	Access Control List
TLS	Transport Layer Security
REST	Representational State Transfer
API	Application Programming Interface
JSON	JavaScript Object Notation
JWT	JSON Web Token
TSDB	Time Series Database
ADC	Analog to Digital Converter
SAR	Successive Approximation Register
PWM	Pulse Width Modulation
GPIO	General Purpose Input Output
MCU	Microcontroller Unit
FSM	Finite State Machine
ISR	Interrupt Service Routine
NVS	Non-Volatile Storage
EMF	Electromotive Force
MAE	Mean Absolute Error
RMSE	Root Mean Square Error
EWMA	Exponentially Weighted Moving Average
CUSUM	Cumulative Sum

Chapter 1

INTRODUCTION

1.1 Motivation and problem statement

Household water management in Vietnam still relies largely on a mechanical float placed inside the storage tank. The device is cheap and durable, but it solves exactly one problem: cutting the pump when the tank is full. The owner has no idea how much water was consumed, cannot notice a pipe leak until the bill rises unexpectedly, and has no way of knowing that the pump is running dry until it has already failed.

The problem carries every characteristic of a cyber physical system, meaning a system in which computation, communication and a physical process are tied together through feedback loops [1]. A physical quantity is measured, digitised, transported to a data infrastructure, turned into a decision, and finally acted back upon the same physical process through an actuator.

From an engineering point of view the tank is a nonlinear plant carrying a hard safety constraint. Discharge flow depends on the square root of the water column height, so the plant response changes with the current level. At the same time there is one constraint that may never be violated under any circumstance, including a manual command issued by an operator: the tank must not overflow. Having to guarantee control quality and an absolute safety constraint at the same time is what lifts this project above a simple exercise in wiring a sensor to a display.

The system must therefore address four groups of problems: measuring level and flow with an error budget established experimentally rather than copied from a datasheet; holding the level inside a desired band without letting the pump chatter around the switching threshold; detecting abnormal hydraulic operating modes; and providing remote supervision through a time series store and a real time interface. Overarching all of them is a single requirement that shapes the whole architecture: essential level control must keep working when the network link is lost.

1.2 Scope and working method

The system is built at laboratory prototype scale with one sensing node, a tank of roughly ten litres and a low voltage direct current pump. The entire electrical side operates at safe extra low voltage.

Distributed multi node deployment, machine learning based consumption forecasting and integration with a commercial cloud platform are outside the scope of this work.

The group organised the work along the V model introduced in the project management part of the course [1], in which every decomposition step on the descending branch has a matching verification step on the ascending branch: requirement analysis pairs with full system acceptance, architectural design pairs with integration testing, protocol specification pairs with protocol level testing, and circuit design pairs with unit testing of each individual sensor.

One methodological decision deserves emphasis. The group specified the complete MQTT topic tree, the message payloads, the set of API endpoints and the database schema before writing a single line of code. That specification acted as an internal contract, which allowed the firmware branch, the backend branch and the interface branch to advance in parallel instead of waiting on one another. While components had not yet arrived, a simulator publishing messages that conform to the agreed specification let the other two branches be developed and tested in full.

Chapter 2

THEORETICAL BACKGROUND AND SYSTEM REQUIREMENTS

2.1 Mandatory requirements of the assignment

Table 2.1.1: Mapping of the mandatory requirements onto the report.

Requirement	Where it is addressed
ESP32 class microcontroller, level sensor and flow sensor	Section 3.3
Low voltage pump behind a driver stage with inductive protection	Section 3.4
Overflow prevention and safe behaviour on implausible readings	Section 6.4
Instantaneous flow rate and accumulated volume	Section 6.2
Publish telemetry, actuator state, faults and availability over MQTT	Chapter 4
Time series storage, REST API and dashboard	Chapter 5
Automatic control using a hysteresis band or a state machine, and at least one fault detection rule	Sections 6.3 and 6.4, seven rules implemented
Manual mode must not override safety, commands acknowledged, local control survives an outage	Sections 6.5, 4.3, 3.2 and 7.3
Advanced component: leak detection using a rolling baseline	Section 6.6
Calibration against a physical reference, control response, fault delay and volume error	Sections 8.2 and 8.3
Sensor fault testing, outage testing and latency quantification	Section 8.4

2.2 Layered architecture and computation placement

An Internet of Things deployment is commonly described by a five layer model made of perception, transport, processing, application and business layers, allowing each stage of signal conversion, transport and storage to be optimised independently [1]. Recommendation ITU-T Y.2060 adds two capability groups that cut across the stack, management and security [2]. Treating security as cross cutting rather than as a separate layer has a direct consequence for our design: it cannot be handled at a single point but must appear in broker authentication, per topic access control, transport encryption and API authorisation. Data flow is likewise split into a southbound path carrying commands down to the hardware and a northbound path carrying telemetry and alerts up to the server [1].

Deciding where a computation belongs is driven by quantitative constraints rather than taste. Edge computation offers deterministic latency and very low bandwidth cost but only short term storage, while cloud computation offers durable storage and analytic capacity at the price of latency from hundreds of milliseconds to several seconds [1]. The resulting rule is that any operation requiring an immediate safe stop must be computed at the edge, whereas trend analysis is routed upward. Cutting the pump on a dry run condition or when the level passes the safety threshold clearly belongs to the first category, which is the direct basis for the decision in Section 3.2.

2.3 Sensing, digitisation and actuation

Faithful reconstruction of a continuous analog signal requires a sampling rate greater than twice its highest frequency component [3]. With the pump actually fitted and the tank base measuring 10 by 10 centimetres, the group measured the fill rate directly rather than deriving it from the datasheet. The level rose 1.2 centimetres in 20 seconds, that is 0.06 centimetres per second, equivalent to 0.36 litres per minute. This is nearly five times slower than the 1.67 litres per minute the JT-DC3L datasheet promises at zero head, the difference being delivery head and tubing resistance. A sampling period of 200 milliseconds therefore satisfies the condition by roughly three orders of magnitude. The group records both figures deliberately: an earlier draft quoted 0.6 centimetres per second from an unchecked estimate, the datasheet implies 0.28, and the bench measures 0.06. Only the last of the three was used to size the timing thresholds of Section 6.3, because a threshold derived from a datasheet the hardware does not obey will fire during normal operation. The ESP32 digitises voltage using a successive approximation register converter [1, 4], and mapping a continuous range onto finite integer steps introduces an unavoidable loss of precision with a quantisation step of

$$\Delta V = \frac{V_{ref}}{2^N} \quad (2.1)$$

and quantisation noise bounded within $\pm \frac{1}{2}\Delta V$. This error is mathematical in nature and can never be filtered away in software [1]. At 12 bits and 3.3 volts the step is 0.806 millivolts, a figure used in Section 3.3. The lectures also warn that the transfer curve is severely nonlinear near both ends of the

range, which is why our dividers place each signal near the middle of the span.

A microcontroller operating at 3.3 volts with roughly 12 milliamperes per pin cannot perform mechanical work directly, so a power stage with galvanic separation has to be inserted. When current through an inductive load is interrupted abruptly, the collapsing field produces a reverse spike following Lenz law, $V = L di/dt$, which can exceed several hundred volts and destroy the switching device; the standard remedy is a flyback diode placed antiparallel across the load [1].

2.4 Application layer messaging

A standard HTTP header easily exceeds 500 to 1000 bytes, so sending a measurement of a few bytes spends most of the energy budget on framing, and every transaction additionally forces the radio through a complete TCP handshake [5]. MQTT addresses this with a binary frame whose fixed header is two bytes, routed through a broker under a publish and subscribe model that decouples communication in space, time and synchronisation [5, 6].

Three MQTT mechanisms are exploited directly by our system. The three quality of service levels define delivery as at most once, at least once and exactly once, and the lectures caution against defaulting every topic to the highest level because a constrained node will exhaust its memory. The last will mechanism lets the broker publish a pre registered payload by itself once a client vanishes without a clean disconnect, and retained messages make the broker push the current value immediately to a newly subscribing client, eliminating the cold start gap [5]. On the security side, MQTTS carries the same binary frames inside a TLS tunnel on port 8883 instead of the unencrypted port 1883, and the lectures stress that a username and password pair alone is insufficient without an access control list mapping each client to the topics it may read and write [5, 7].

2.5 Data infrastructure and the feedback sync principle

Machine telemetry has three structural invariants: it is append only, indexed primarily by time, and its write to read ratio is very high [8]. A relational engine is built for transactional integrity, so every inserted row forces a rebalancing of an in memory B-tree, whereas a time series engine optimises for tail appends and separates indexed tags from unindexed fields. The lectures also warn about index cardinality explosion when a randomly varying value is placed in a tag position [8], a principle our schema in Section 5.2 follows. The REST style demands client server separation and statelessness, favours nouns over verbs, and asks the designer to evaluate safety $f(s) = s$ and idempotency $f(f(s)) = f(s)$ [9, 10]; on the microcontroller side, firmware must forbid dynamic allocation inside the loop and reserve a fixed buffer at startup [10, 11].

The single idea with the largest influence on our interface design is the feedback sync principle derived from the device shadow model [8]. The anti pattern to avoid is a dashboard toggle that flips to the on state the instant the user presses it, before anything is known about whether the real machine started.

The correct architecture splits the path: the user action maps to an outgoing command topic while the indicator stays unchanged until an explicit acknowledgement arrives from the device. This is the basis for the mechanism described in Section 4.3.

2.6 On off control with hysteresis

The material in this section does not appear in the course slides. The group added it from control engineering literature and states the sources here explicitly, so that it is clearly distinguished from the knowledge taken from the lectures.

On off control, also called two position control, is the simplest form of feedback control, in which the actuator takes one of exactly two states [12]. If the two thresholds coincide, even the smallest measurement noise around that point makes the actuator flip repeatedly, causing rapid wear and repeated inrush events. The standard remedy is to separate them into a hysteresis band inside which the state is held unchanged [13]:

$$u(t) = 1 \text{ if } h < h_L; \quad u(t) = 0 \text{ if } h > h_H; \quad u(t) = u(t^-) \text{ if } h_L \leq h \leq h_H \quad (2.2)$$

The width $h_H - h_L$ is a direct trade off parameter: a wider band lowers the switching frequency but widens the level excursion.

For diagnosis, the group applies the principle of checking consistency between two independent signal sources, following established fault diagnosis theory [14]. Slow leaks belong to the class of small persistent shift detection, for which the cumulative sum method of Page is far more sensitive than a fixed threshold because it accumulates small deviations over time [15].

To select control parameters before hardware existed, the group built a dynamic model from the volume balance $dV/dt = Q_{in} - Q_{out} - Q_{leak}$ with $h = V/A$, where discharge follows Torricelli law $Q_{out} = C_d A_o \sqrt{2gh}$ [16]. The square root dependence makes the plant nonlinear, since discharge at a full tank is appreciably faster than near empty. This is why the hysteresis parameters had to be swept in simulation, as described in Section 8.4.

Chapter 3

ARCHITECTURE AND HARDWARE DESIGN

3.1 Overall architecture

The system consists of four blocks in series: an edge node built around an ESP32 with three sensors and one actuator; an MQTT broker acting as a content based message router; a backend service comprising an ingestion process, a database and an API server; and a web dashboard.

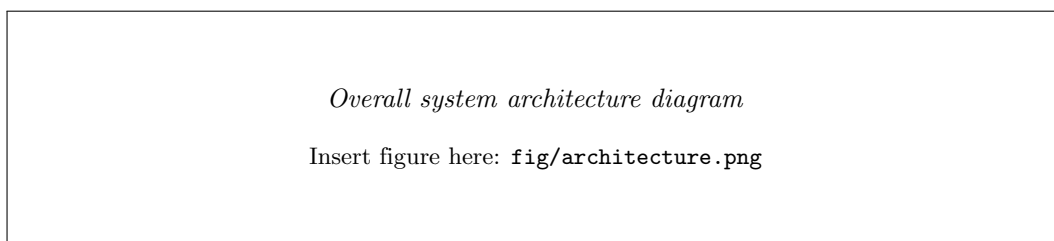


Figure 3.1.1: Architecture of the smart water tank system.

Mapping onto the five layer model of Section 2.2: the perception layer is the ESP32 with its sensors and relay; the transport layer is a Mosquitto broker over Wi-Fi; the processing layer is the ingestion process, SQLite and a FastAPI server; the application layer is the dashboard; and the business layer, kept deliberately minimal at this scale, is the daily consumption report. The single most important property of the diagram is that the closed control loop lives entirely inside the first block, so the remaining three never sit on the path of a pump switching decision.

Wi-Fi was selected over BLE, Zigbee and LoRaWAN for three reasons. The system is mains powered, so the low energy advantage of BLE and LoRaWAN carries no weight. The device sits indoors within a few tens of metres, comfortably inside Wi-Fi coverage. Most importantly, Wi-Fi gives the node a native IP address so it speaks to the broker directly, whereas every alternative needs an edge gateway to translate between two protocol worlds [17], adding hardware and a point of failure. The conclusion reverses for tanks spread over a wide area or for a battery powered node.

3.2 Function placement and edge autonomy

This is the central architectural decision of the project. The assignment requires that level control keep working during a network outage. Combined with the placement rule of Section 2.2, which states that any operation needing an immediate safe stop must be computed at the edge [1], the group adopted a single principle: **the server never issues a direct pump switching command**.

The edge node therefore owns sensor reading and filtering, the control state machine, every safety interlock, all fault detection rules and volume accumulation, because each of these demands a deterministic cycle and has to survive an outage. The backend owns only the two things the node cannot do well, namely durable history for querying and statistical baseline leak detection, which needs a window of several hours and exceeds the memory available on the microcontroller.

This split makes the system degrade in a controlled way when connectivity is lost: the user loses remote visibility and remote commands, but the tank is still refilled at the correct thresholds and every safety constraint is still enforced. The behaviour is verified experimentally in Section 8.4. So that leak diagnosis is not lost entirely during an outage, the node also runs a simpler leak rule based on consistency between pump state and measured flow, described in Section 6.6.

3.3 Sensor selection and signal conditioning

Table 3.3.1: Main bill of materials.

Component	Role
ESP32 DevKit V1	Edge microcontroller
HC-SR04 ultrasonic, Hall turbine flow, ACS712	Level, flow rate and current presence sensing
Two mechanical float switches, both on the storage tank	Upper level overflow guard and lower level fill permission
Single channel opto isolated relay, 5 V submersible pump	Switching stage and actuator
1N4007 diode, 100 nF ceramic, 1000 μ F electrolytic	Inductive protection, brush noise filtering, inrush buffering
10 k Ω and 20 k Ω resistors, 4.7 k Ω pull ups, 5 V 3 A supply	Two divider networks, open collector pull ups and system power

Level sensing. Non contact ultrasonic measurement was chosen because the transducer never touches the water, so it neither corrodes nor accumulates deposits, and its output does not depend on water conductivity. Its main constraint is a blind zone of roughly 20 to 25 centimetres, so the bracket is mounted 30 centimetres above the tank rim.

Flow sensing. Pulse frequency is proportional to flow through a factor K as $Q = f/K$. The datasheet

value only holds near the middle of the measuring span, so the group determines K experimentally in Section 8.2. One risk was identified at design time: the 5 volt pump delivers roughly 1.5 litres per minute while typical turbine sensors begin measuring at 1 litre per minute, which places the operating point near the lower limit, exactly where nonlinearity is worst. Mitigations include shortening the tubing, reducing the lift height and mounting the sensor vertically with upward flow.

Float switches. The two floats are not there to measure anything. The upper float forms an overflow guard that operates independently of software and, more importantly, supplies a level indication that is entirely independent of the ultrasonic path. Because two independent sources exist, the firmware can detect a contradiction between them and conclude that the ultrasonic reading is wrong. This is how the system satisfies the requirement to handle implausible sensor readings, as detailed in Section 6.4.

Divider networks. All three signal lines exceed the 3.3 volt logic limit of the ESP32 and therefore pass through resistive dividers. The ultrasonic echo and the flow pulse lines use a 10 k Ω and 20 k Ω ratio, bringing 5 volts down to 3.33 volts. The current sensor line uses a one to one ratio, moving its no load operating point from 2.5 volts to 1.25 volts, that is to the middle of the span, avoiding the two nonlinear regions the lectures warn about [1].

Limits of the current measurement. The sensor provides 185 millivolts per ampere and the pump draws roughly 200 milliamperes, giving only 37 millivolts, reduced to 18.5 millivolts after the divider, while the quantisation step from equation (2.1) is 0.806 millivolts and real converter noise is several times larger. The measurement therefore lacks the resolution for a trustworthy quantitative value, so it is used only as a binary current present indicator obtained by averaging 200 samples and subtracting a startup offset. Stating this openly, instead of publishing a figure that cannot be trusted, was a deliberate choice.

3.4 Power stage, supply and mechanical build

The pump is switched through an opto isolated relay module. A power MOSFET was considered, since the lectures favour it in edge designs of modest power thanks to its very low on resistance [1], but the relay was chosen because the module already integrates optical isolation between logic and load, and because a pump running in two states at a low switching rate never exercises the frequency advantage of a MOSFET. Inductive protection uses a 1N4007 diode antiparallel across the pump terminals, alongside a 100 nF ceramic capacitor at the motor terminals whose purpose is not spike suppression but damping of brush noise that would otherwise propagate along the supply rail and disturb the ultrasonic measurement.

The whole system runs from a single 5 volt 3 ampere supply. The issue to manage is the transient sag at pump start, since if the microcontroller supply falls below the critical threshold the integrated brownout detector puts the chip into reset [1]; the remedy is a 1000 microfarad capacitor near the supply pin to buffer the inrush peak.

The rig forms a closed water loop of a source bucket, the pump, an inlet flow sensor, a transparent main tank with a millimetre scale for calibration, a manual valve acting as the consumption load, and a second flow sensor on the outlet. Two sensors rather than one is a deliberate choice: a single inlet sensor measures only what the pump delivers, so consumption has to be inferred from the level derivative, which

is noisy at low draw rates. Measuring both ends makes inflow and outflow independent observations and opens a water balance, where the difference between the two should account for the change in stored volume; a persistent mismatch is evidence of a leak that neither sensor alone would reveal. The two sensors carry separate calibration constants because they need not be the same model. The sensor bracket must be rigid, because vibration makes readings oscillate and would be misdiagnosed as a sensor fault. The circuit was assembled in seven steps, each with its own test program that had to pass before moving on.

Chapter 4

PROTOCOL AND COMMUNICATION

4.1 Design principles and topic tree

Three principles govern the protocol specification. First, every message carries a device generated timestamp, because although TCP preserves ordering on a single socket, brokers process multiple threads concurrently so final arrival order may differ, and deterministic ordering must therefore be imposed at application level inside the payload itself [5]. Second, the quality of service level is chosen per topic according to the consequence of losing a message. Third, every downward command must be answered by an upward acknowledgement carrying the same identifier.

The topic root is built from broad scope down to the individual device as `wt/lab1/esp32-01`, following the hierarchical naming rule given in the lectures [5].

Table 4.1.1: MQTT topic tree and the reasoning behind each setting.

Topic	QoS	Retain	Reasoning
<code>.../telemetry</code>	0	No	One lost sample in a continuous 1 Hz stream changes no conclusion about the trend, and in exchange latency is lowest with no device buffer
<code>.../state/pump</code>	1	Yes	Published only on change, so a loss leaves the dashboard wrong until the next transition
<code>.../event/fault</code>	1	No	Losing an alarm is incorrect behaviour for a safety function
<code>.../status</code>	1	Yes	The last will payload is published once, with no retransmission
<code>.../cmd</code> and <code>.../cmd/ack</code>	1	No	A lost command or acknowledgement is a direct functional failure

The design respects all three naming pitfalls listed in the lectures [5]: no leading separator, no whitespace inside the path, and six explicit subscriptions rather than a root wildcard. No topic uses the highest quality of service level, because it requires a four step handshake and packet identifier tracking on the edge device, while the duplicate delivery problem it solves is already handled at application level: each command carries a unique identifier, so a duplicate simply causes the device to resend the same acknowledgement without acting twice. This is the idempotency property of Section 2.5.

4.2 Message structure and fault codes

Payloads use JSON built with a statically allocated document, following the rule that forbids dynamic allocation inside the loop [10, 11].

```
{ "dev":"esp32-01", "ts":1735689600, "ts_ms":1735689600123, "seq":10421,
  "level_pct":62.4, "level_cm":15.6, "level_ok":true,
  "flow_lpm":1.28, "volume_l":143.62, "volume_today_l":12.40,
  "pump":true, "state":"FILLING", "mode":"AUTO",
  "current_mv":38.2, "float_max":false, "float_min":true, "rssi":-55 }
```

Listing 4.1: Telemetry message payload.

The `seq` field is a monotonically increasing counter allowing lost messages to be counted exactly in Section 8.4. The `ts` field is produced by the device after time synchronisation and supports latency measurement. The `level_ok` field states whether the reading can be trusted, letting the interface distinguish a genuinely low tank from a broken sensor. Messages replayed after reconnection carry a `replay` flag and are excluded from latency statistics.

Eight fault codes are defined and fall into two groups. The non self clearing group, made of `DRY_RUN`, `NO_CURRENT`, `FILL_TIMEOUT` and `OVERFLOW`, requires the operator to confirm that the physical cause has been dealt with before the fault can be cleared. The self clearing group, made of `SENSOR_TIMEOUT` and `SENSOR_CONFLICT`, concerns signal quality and resolves on its own once the signal has been valid for long enough. The remaining two, `LEAK_SUSPECTED` and `LEAK_BASELINE`, raise a warning without locking the pump. Trigger conditions for each code are given in Section 6.4.

4.3 Command and acknowledgement mechanism

This section implements the feedback sync principle of Section 2.5 and satisfies the requirement that commands be acknowledged with a verified state.

```
// .../cmd
{"cmd_id":"a7f3c2e1","action":"pump","value":true,"ts":1735689600}
// .../cmd/ack
{"dev":"esp32-01","cmd_id":"a7f3c2e1","status":"rejected",
  "reason":"overflow_guard","state":"OVERFLOW_LOCK","pump":false}
```

Listing 4.2: Command message and matching acknowledgement.

The acknowledgement reports not only the outcome but also the actual device state after processing, so the dashboard always displays a state the device has confirmed rather than one the user has just requested. Four actions are supported: change mode, switch the pump, clear a fault and reset the daily volume counter. Ten rejection reasons are defined, including `auto_mode_active`, `fault_active`, `overflow_guard`, `float_max_triggered`, `level_sensor_invalid` and `min_off_time`.

The rejection table is worth more than a debugging aid: during the demonstration, pressing the pump on button while a fault is active and seeing an explicit rejection with a named reason is visible proof that manual mode cannot override the interlocks.

4.4 Connection lifecycle and availability monitoring

The firmware implements the four state connection machine described in the lectures. The critical point is the accompanying design note: connection logic must never sit inside a blocking wait loop, because that stalls the processor and suspends both analog sampling and safety checks for the whole outage [5]. Our reconnection routine therefore never blocks, retries only once its interval has elapsed, and the control loop runs independently of network state. The retry interval grows exponentially with an upper bound, $T_{retry} = \min(T_{max}, T_{base} \cdot 2^n)$ with T_{base} of 1 second and T_{max} of 30 seconds [18].

On connecting, the device pre registers a last will message announcing that it is offline, then overwrites it with an online message once the session is established. Should the node suffer an abrupt loss of power, the broker keep alive supervisor publishes the stored will by itself [5], so the interface switches to a disconnected state with no polling logic on the application side. A second backend check treats the device as offline when the newest telemetry message is older than 5 seconds. The two complement each other, since the will reacts quickly to a total power loss while the freshness check also catches a live session whose data stream has stopped.

Chapter 5

BACKEND, DATA MANAGEMENT AND INTERFACE

5.1 Backend architecture

The backend is a single Python process containing three concurrent components. The ingestion component holds a persistent broker session and subscribes to the six specified topics. The API server exposes endpoints for the dashboard and for extracting experimental data. The rolling baseline leak detector is invoked on every incoming telemetry message and is described in Section 6.6. In addition the process keeps an in memory copy of the latest state, so that the current state endpoint can answer without touching the database, since the dashboard calls it once per second.

For every telemetry message the ingestion component records the device timestamp alongside `recv_ts`, written by the backend on arrival, so that a one way delay $\tau = t_{recv} - t_{sample}$ is available from a single query without external instrumentation. The device timestamp is stored twice: `ts` at one second resolution, kept so that existing consumers of the payload continue to parse, and `ts_ms` at millisecond resolution, which is the field the latency query prefers. The coarser field alone carries a quantisation error of up to 500 milliseconds, larger than the transport delay it is meant to measure, and Section 8.4 records the corrected figures side by side. The measurement assumes both clocks are synchronised; messages emitted before synchronisation completes carry a zero timestamp and are excluded from the statistics. Because that assumption cannot be discharged on a microcontroller disciplined only by the network time protocol, the one way figure is treated as a comparison quantity and the command round trip, timed entirely by the server clock, is the reported result.

5.2 Data schema and API

The schema has six tables, split according to data behaviour: **telemetry** is written continuously at one row per second, while **faults**, **pump_events**, **commands**, **availability** and **config** are written sparsely and discretely. The telemetry table is indexed on the timestamp column, the only column ever used as a filter predicate. The group applies the tag and field separation principle of Section 2.5: the device identifier acts as an indexed tag while the measured quantities are unindexed fields. In particular the sequence number column, although it changes on every row, is deliberately left unindexed to avoid exactly the cardinality explosion the lectures warn about [8]. The command table stores both the send time and the acknowledgement time, whose difference is the command round trip latency.

On database choice, the group is aware that the theory in Section 2.5 identifies real limits of relational engines for high rate time series data [8]. Those limits, however, appear at thousands of writes per second, whereas this system writes one row per second from one node, three to four orders of magnitude below the problematic regime. Deploying a dedicated time series engine would add operational complexity without any measurable benefit, so the group chose SQLite while still respecting the schema design principles above [19].

Table 5.2.1: Main API endpoints.

Method	Path	Purpose
GET	/api/latest	Latest state, data freshness, pending commands
GET	/api/telemetry	Time series over a requested interval
GET	/api/volume/daily	Consumed volume per day
GET	/api/faults, /api/events, /api/commands	Fault log, pump switching log, command log with acknowledgement latency
POST	/api/command	Issue a command, returns its identifier
GET, PUT	/api/config	Read and update threshold configuration
GET	/api/stats/latency	Latency statistics used in Section 8.4

Method selection follows the two properties introduced in Section 2.5. Read endpoints use GET because they are safe and change no state. The configuration endpoint uses PUT because replacing the full threshold set is idempotent. The command endpoint uses POST because each call creates a new command record with its own identifier and is therefore not idempotent [10].

5.3 Monitoring interface

Dashboard screenshot while the pump is running

Insert figure here: `fig/dashboard.png`

Figure 5.3.1: Monitoring dashboard in automatic mode.

The interface is designed for the situation in which an operator glances at the screen for a few seconds to answer three questions: what is the level, is the pump running, and is anything wrong. The left column is the state block, holding a tank drawing with two horizontal lines marking the low and high thresholds drawn at their true positions for the active configuration, the numeric readouts and the control block. The right column is the history block with two time series charts and two log tables. Drawing the thresholds directly on the tank lets a viewer see immediately where the level sits relative to the hysteresis band and understand why the pump turned on or off at that instant.

On the update mechanism, the lectures recommend REST for low frequency administrative tasks and WebSocket for live streams [18, 20]. The group chose polling at 1 Hz, which runs against that general recommendation and therefore needs justification. The device publishes at 1 Hz, so the polling rate equals the data production rate and no request is wasted. With one client and one node the resulting traffic is a few kilobytes per second, negligible on a local network. A WebSocket, by contrast, introduces a stateful channel with its own lifecycle to manage, including heartbeats and reconnection [18], that is a new set of failure states in exchange for a benefit that is not measurable at this scale. The group states the point at which the conclusion reverses: beyond roughly ten nodes, or beyond a required update rate of 5 Hz, moving to a WebSocket becomes worthwhile.

The feedback sync principle is implemented in how command state is displayed. When the user presses the pump on button the interface does not change the icon; it posts the request, receives an identifier and shows a pending acknowledgement line. The icon changes only on the next update cycle, once the pump state field in the telemetry has genuinely changed. The command log shows the action, the outcome, the rejection reason and the acknowledgement latency in milliseconds, which is useful operationally and also the direct data source for Section 8.4.

The interface adapts its presentation to the state reported by the device, following the conditional rendering principle [18]: data older than 5 seconds or an arriving last will message turns the connection indicator red; a fault or overflow lock state switches the status label to an alarm style; and a false `level_ok` flag marks the level value as untrusted. Control buttons are disabled contextually, but this is only a convenience layer, since the real interlock lives in the microcontroller and is described in Section 6.5.

5.4 Device simulator

The simulator implements the hydraulic model introduced in Section 2.6 together with a sensor noise model, and publishes messages that conform to the specification of Chapter 4. It serves two purposes: while components had not arrived it allowed the backend and interface branches to be developed fully, and it allows control parameters to be swept across dozens of configurations in a few minutes, which is not feasible on real hardware. The sweep results are reported in Section 8.4. The simulator can also inject three failure modes, namely a pump that runs without delivering water, a sensor stuck at a fixed value, and a leak at a prescribed flow rate.

Chapter 6

CONTROL AND SYSTEM INTELLIGENCE

6.1 Overview

This chapter presents the main technical contribution of the project. All of it runs in the microcontroller firmware, with the single exception of the statistical baseline leak detector of Section 6.6, which sits on the backend for the reason given in Section 3.2. The control loop runs on a fixed 200 millisecond cycle consisting of four sequential steps: read and filter sensors, evaluate the fault detection rules, run the state machine, and update the outputs. The loop does not depend on network state.

6.2 Computed quantities

Water column height is derived from the measured distance and then corrected as $h_{corr} = a \cdot h_{meas} + b$, where the two coefficients come from the linear regression performed in the calibration experiment of Section 8.2.

Flow sensor pulses are counted by a hardware interrupt on the falling edge. The handler does exactly one thing, incrementing a counter declared `volatile` so the compiler does not optimise away repeated reads. When the control loop reads it, interrupts are briefly masked for the duration of the copy to avoid a race on a multi byte variable, in line with the thread safety warning in the lectures [1]. Accumulated volume is the numerical integral of flow:

$$V(t) = \sum_i Q_i \cdot \frac{\Delta t}{60} \quad (6.1)$$

The accumulation runs on the microcontroller rather than the server, because if it ran on the server every message lost during an outage would become water permanently unaccounted for. The value is written to non volatile storage every 60 seconds, sparse enough not to wear the flash yet dense enough that a sudden power loss costs at most one minute of operation.

Noise filtering. The water surface ripples strongly while the pump is filling the tank, so the raw

reading sequence contains isolated large amplitude outliers. The group applies a five sample median filter, choosing the median over the arithmetic mean because a single outlier drags the mean but leaves the median untouched. In addition the firmware rejects a sample at source if the reading falls outside the physical range of the tank or changes faster than 12 centimetres per second, a threshold roughly twenty times the real rise rate so genuine data is never discarded.

6.3 State machine and hysteresis

Table 6.3.1: Controller state set.

State	Entry condition	Exit condition
BOOT	Power up	A valid level reading is available
IDLE	Sufficient water, pump off	Level below the low threshold and minimum off time elapsed
FILLING	Pump started	Level above the high threshold and minimum on time elapsed
MANUAL_ON	Manual command accepted	Return to automatic or an interlock violation
FAULT_DRYRUN	Dry run rule triggered	Operator clears the fault
FAULT_SENSOR	Sensor rule triggered	Valid signal for 5 continuous seconds
OVERFLOW_LOCK	Level past the safety limit or float engaged	Fault cleared and level has dropped

The reason for a state machine rather than a plain conditional expression is that fault states need to be sticky. A conditional rule would immediately allow the pump to restart the moment a transient fault condition disappears, for instance when the suction tube accidentally touches the water again; the state machine holds the system in the fault state until an explicit intervention occurs.

The transition between `IDLE` and `FILLING` implements equation (2.2) with two separated thresholds plus two timing constraints. The low threshold of 30 percent and the high threshold of 70 percent come from the parameter sweep of Section 8.4, and additionally leave a 15 percent margin up to the 85 percent safety limit to absorb overshoot. The minimum on time of 3 seconds protects the pump from repeated start cycles, the minimum off time of 20 seconds lets the supply voltage settle, and the maximum run time of 200 seconds, roughly twice the measured fill duration, serves as an abnormality threshold.

The minimum on time deserves a note, because the first draft set it to 10 seconds and that value was unsafe. The working volume of this tank is 1 litre, and at the measured 0.36 litres per minute a forced 10 second run adds 0.06 litres, 6 percent of the tank. Holding the pump on for that long while the state machine has no authority to stop it is a guarantee of overshoot past the high threshold. Three seconds is long enough to stop the relay chattering and short enough that the band still governs.

A hysteresis width of 50 percent may look wide for a conventional control system. The choice follows from the nature of the application: the tank is a buffer, and the goal is not to hold the level at a precise value but to guarantee water availability while limiting the number of pump starts. The two timing constraints complement the band: hysteresis rejects measurement noise near the threshold, while the timing constraints reject genuine short period oscillations such as surface waves.

6.4 Fault detection rules

The system implements seven rules where the assignment asks for at least one. They are evaluated in priority order and the first rule that fires ends the chain.

Table 6.4.1: Fault detection rules and trigger conditions.

Code	Trigger condition	Action
SENSOR_TIMEOUT	No valid level reading for 4 seconds	Stop pump, enter sensor fault
SENSOR_CONFLICT	Upper float engaged while ultrasonic reports below 70 percent	Stop pump, enter sensor fault
OVERFLOW	Level above 85 percent, raw echo closer than 4.5 cm, or upper float engaged	Stop pump, lock
DRY_RUN	Pump on for 6 continuous seconds with flow below 0.25 litres per minute	Stop pump, lock
NO_CURRENT	Pump on for over 2 seconds with no current detected	Stop pump, lock
LEAK_SUSPECTED	Pump off yet flow above 0.20 litres per minute for 60 seconds	Warning
FILL_TIMEOUT	Pump running over 200 seconds without reaching the high threshold	Stop pump, lock

Three rules deserve comment. **SENSOR_CONFLICT** is a direct application of the independent source consistency principle of Section 2.6 [14]. With only an ultrasonic sensor the system has no way to tell whether a reading is right or wrong as long as the value stays inside the physically plausible range; the presence of a mechanical float turns a wrong reading into a detectable contradiction. This is the real engineering reason for fitting the floats, beyond their overflow role.

DRY_RUN is the rule explicitly demanded by the assignment. Its 6 second window is long enough to ignore the transient while the pump starts and the pipe has not yet filled, but short enough that the pump never runs dry for long.

NO_CURRENT uses the current signal in the binary form justified in Section 3.3 and separates two distinct causes of the same symptom: current with no flow means the pump is running dry or the line is blocked, whereas neither current nor flow means an electrical fault such as a failed relay or a broken

wire, telling the operator which side to inspect.

6.5 Manual mode and safety interlocks

The assignment requires that manual mode must not override safety constraints. The group meets this with an architectural decision inside the firmware: **every code path capable of starting the pump is forced through one single guard function.** The function returns empty when starting is permitted and a reason code when it is blocked, evaluating six blocking conditions: an unresolved fault, the upper float engaged, a raw echo returning from closer than the safety distance, level past the safety limit, an untrusted level measurement, and the minimum off time not yet elapsed.

An earlier revision listed an empty source tank as a seventh condition. That condition was removed, and the removal is worth recording. The source vessel carries no instrument of any kind, so the firmware has no means of knowing whether it holds water; the condition was in fact reading the lower float of the *storage* tank, which had been mislabelled. Blocking the pump on a quantity the system cannot observe is not a safety measure but a permanent refusal to operate, and dry running is already covered by three rules that rest on quantities the system does observe: absent flow, absent current, and absent progress in the level.

The significance is that no bypass can be created by accident. Whether the request originates from the automatic state machine or from a user command, both are stopped by the same condition set, so verifying that the interlock is correct means reviewing one function rather than auditing the whole codebase. When a command is refused the device does not stay silent but returns an acknowledgement carrying the specific reason code. Moreover, even while in `MANUAL_ON`, the state machine continues to evaluate the safety conditions on every cycle and stops the pump on its own if the level passes the limit or a float engages.

6.6 Leak detection

This is the advanced component required by the assignment, built as two complementary mechanisms at two different places in the system.

The edge mechanism is the `LEAK_SUSPECTED` rule, based on a simple physical observation: once the pump is off, the supply line should carry no flow at all. Its advantage is that it runs locally and therefore survives an outage, with a bounded detection delay; its limitation is that it only sees leaks on the pipe section carrying the sensor and needs enough flow to turn the turbine.

The second mechanism tackles the harder problem of detecting a sustained abnormal consumption level, that is the small slow shift discussed in Section 2.6 [15]. The algorithm divides the day into 48 slots of 30 minutes, and each slot maintains a baseline mean and variance updated by an exponentially weighted moving average:

$$\mu_k \leftarrow \mu_k + \alpha(V_k - \mu_k), \quad \sigma_k^2 \leftarrow (1 - \alpha) [\sigma_k^2 + \alpha(V_k - \mu_k)^2] \quad (6.2)$$

with α set to 0.2. An alert is raised when $V_k > \mu_k + 3\sigma_k$ holds for at least two consecutive slots.

Slicing the day rather than keeping one global baseline has a practical reason: domestic water consumption varies strongly with the hour, so a fixed threshold would be either too sensitive at night or too insensitive at peak time. Requiring two consecutive slots is an accumulation mechanism in the spirit of the cumulative sum method [15], intended to suppress false alarms caused by a single unusual draw such as washing a vehicle.

Chapter 7

SECURITY AND RELIABILITY

7.1 Threat model and mitigations

Under the ITU-T Y.2060 model of Section 2.2, security is a cross cutting capability acting on every layer rather than a layer of its own [2], so the analysis is organised per layer.

Table 7.1.1: Threat scenarios and mitigations.

Attack scenario	Consequence	Mitigation
Connect to the broker and publish a pump on command	Overflow, pump damage	Mandatory authentication, per topic access control
Sniff traffic to harvest credentials	Full control takeover	TLS transport on port 8883
Forge telemetry so the system mis-reads the level	Incorrect pumping, possible overflow	Local interlock based on the mechanical float
Replay commands repeatedly to make the pump chatter	Mechanical pump damage	Minimum on and off times enforced on the device
Call the API directly, bypassing the web interface	Unauthorised commands	Token authentication, administrator and viewer roles

The two middle rows are worth highlighting because they are mitigated not by cryptography but by the control design itself. The mechanical float is a physical constraint that no network message can override, and the minimum switching times are enforced in firmware, so an attacker may send any number of commands without making the pump toggle faster than the limit. This illustrates the principle that safety in a cyber physical system cannot be separated from its control design.

7.2 Authentication, authorisation and encryption

The Mosquitto configuration disables anonymous connections entirely and mandates both a password file and an access control list file [21]. The group applies least privilege: the edge device may only publish on the northbound topics under its own branch and may only subscribe to its own command topic, while

the backend may publish on the command topic and read the whole northbound branch. The practical meaning is that if the edge device is compromised, the attacker still cannot use that device to issue control commands, because the device account has no write permission on the command topic. This is exactly the node compromise scenario the lectures raise when introducing access control lists [5].

On the API side the system distinguishes an administrator role, allowed to call the command and configuration endpoints, from a viewer role restricted to read endpoints [10, 22]. The split maps directly onto HTTP status codes: a missing token returns 401 while a valid token with insufficient privilege returns 403.

The default configuration runs on the unencrypted port 1883, which suits an isolated laboratory network. For a real deployment the group switches to MQTTS on port 8883 [5, 7]. The cost should be stated plainly: the TLS handshake requires the microcontroller to perform public key operations, which noticeably increases startup time and memory consumption. One principle holds in every configuration, namely that no secret ever appears inside the telemetry payload, so even a successful eavesdropper gains no credentials.

7.3 Reliability during a network outage

This is a mandatory requirement and also the most important property of the system. Four mechanisms are implemented.

First, the control loop runs on a fixed cycle and contains no blocking network call, so during an outage the system loses only remote visibility while retaining every control and safety function.

Second, when the link is down telemetry messages are not discarded but packed into a compact structure and pushed into a circular buffer holding 240 entries, about four minutes of data. The circular buffer was chosen for exactly the reason given in the lectures, namely constant time insertion with no array shifting [1]. Each entry stores only the essential fields as scaled integers instead of a full JSON string. Once the link returns the whole buffer is replayed in order, tagged so it can be excluded from latency statistics.

Third, accumulated volume is written to non volatile storage and reloaded at startup, so a sudden power loss does not erase the consumption history.

Fourth, inside the initialisation routine and before any sensor is read, the relay control pin is driven to the pump off level, so every restart begins from a safe state. The integrated brownout detector acts as an additional protective layer: when the supply falls below the critical threshold, flash writes become unreliable and the core may execute corrupted instructions, so the hardware deliberately holds the device in a safe reset state until the voltage recovers [1].

Table 7.3.1: Four independent layers of overflow protection.

Mechanism	Location	Still effective when
High threshold of the hysteresis band	State machine	Sensor is healthy
Safety limit at 85 percent	Fault detection rule	Sensor is healthy
Raw echo closer than 4.5 cm	Fault detection rule	Median filter is producing nonsense
No level progress in 45 seconds	Fault detection rule	Level sensor reports a frozen or false value
Unconditional 240 second run limit	Fault detection rule	Every sensor has failed
Upper mechanical float	Digital input of the microcontroller	Ultrasonic sensor has failed
Relay defaults to open	Hardware	Microcontroller loses power or hangs

The value of this arrangement lies in the independence of the layers. The first two both rely on the ultrasonic sensor and therefore fail together if it fails, but the third uses an entirely different sensing principle and the fourth requires no running software at all. Consequently no single point of failure can disable the overflow protection as a whole.

Chapter 8

EXPERIMENTS, RESULTS AND CONCLUSION

8.1 Principles and reference instruments

Three principles apply to the whole experimental programme: every measurement must have a reference standard independent of the system under evaluation; every experiment must be repeated enough times to yield a dispersion figure and not merely a mean; and results must be reported through standardised statistical indicators rather than qualitative remarks [23, 24]. The reference instruments are a millimetre scale attached to the tank wall, an electronic balance with one gram resolution, a multimeter, and a system clock synchronised over the network. The balance was chosen as the volume reference because at room temperature the density of water is close enough to one kilogram per litre that the conversion error is far smaller than the error of the sensor being evaluated. The indicators used throughout are the per point error $e_i = x_{meas,i} - x_{ref,i}$, the mean absolute error $MAE = \frac{1}{n} \sum |e_i|$ and the root mean square error.

Table 8.1.1: Plan of the eight experiments.

ID	Subject	Repeats	Reported result
E1	Level calibration	10 points	MAE, RMSE, regression coefficients
E2	Flow calibration	5 runs	Factor K , deviation from datasheet
E3	Volume error	5 runs	Relative error of two estimation methods
E4	Control response	5 runs	Fill time, overshoot
E5	Dry run detection delay	10 runs	Mean, standard deviation, false alarms
E6	Sensor faults and leaks	5 leak rates	Smallest detectable leak rate
E7	Latency and outage	5 runs	Median, 95th percentile, lost messages
E8	Control policy comparison	3 runs	Reduction in switching count

Note. The result tables below are intentionally left blank. They must be filled with data measured on the physical prototype, never with estimates or with figures taken from simulation.

8.2 Sensor calibration

E1. Fill the tank in steps, stopping at ten levels evenly spread from empty to full. At each level read the reference value from the scale and record three dashboard readings five seconds apart. A linear regression between reference and measured values yields the two coefficients loaded into the firmware; after loading them, three arbitrary points are remeasured to confirm the error has fallen.

E2. Place the collecting vessel on the balance and tare it, run the pump for exactly 60 seconds, then record the collected mass and the pulse count, repeating five times. The real factor is $K_{meas} = N_{pulses}/(60V_{ref})$ and the deviation from the datasheet is $\delta_K = (K_{meas} - K_{nom})/K_{nom}$.

Table 8.2.1: Calibration results for the two main sensors.

Indicator	Before calibration	After calibration
E1: mean absolute error (cm)		
E1: root mean square error (cm)		
E1: slope a and offset b		
E1: coefficient of determination and worst repeatability		
E2: measured factor K		
E2: deviation δ_K from datasheet (%)		

Discussion points. For E1 the report must state whether the error is spread evenly across the span or concentrated in one region, and whether repeatability is small enough relative to the hysteresis width, since if the two are comparable the control design loses its effect. For E2 the essential content is explaining the cause of the deviation: as analysed in Section 3.3 the operating point sits near the lower limit of the measuring span, so a large deviation is a predictable outcome rather than an experimental mistake.

8.3 Control response and fault detection

E3. Record the accumulated volume and the initial level, run the pump for 90 seconds, then record all three figures: the volume integrated by the system, the volume derived from the level rise as $V = A\Delta h/1000$, and the volume weighed on the balance. Beyond quantifying the error, this experiment validates the very assumption the leak rule depends on: if the two estimation methods agree under normal conditions, an unusual divergence between them is a trustworthy leak indicator.

E4. Open the drain valve until the level falls below the low threshold, then let the system run one

full fill cycle and export the data from the time series endpoint. Record the level at pump start and stop, the fill duration, and the overshoot as $h_{max} - h_H$. Overshoot reflects the inertia of the plant, since water remaining in the pipe keeps entering the tank after the stop command; this is why the high threshold sits at 70 percent rather than close to the safety limit.

E5. While the pump is running, lift the suction tube out of the water. For accuracy the group does not use a stopwatch but takes the difference between the timestamp of the fault record and the timestamp of the last telemetry message still showing normal flow. In parallel, run the system normally for 30 minutes and count false alarms; this figure matters as much as the delay, because an over sensitive detector makes operators lose trust and ignore alerts.

E6. Part A creates three sensor fault situations: unplug the ultrasonic signal wire, engage the upper float by hand while the tank is nearly empty, and place an obstacle close to the transducer. Part B slightly opens the drain valve at five different flow rates while the pump is off, holding each for three minutes.

Table 8.3.1: Control response and fault detection results.

Quantity	Result
E3: relative error of the flow integration method (%)	
E3: relative error of the level based method (%)	
E4: actual pump start and stop levels (%)	
E4: mean fill time and mean overshoot	
E5: dry run detection delay, mean and standard deviation (s)	
E5: false alarms over 30 minutes	
E6: three sensor fault rules fired correctly	
E6: smallest detectable leak rate (L/min)	
E6: precision P and recall R	

Discussion points. Compare the actual start and stop levels against the configured thresholds, since the gap reflects the delay of the sensing and processing chain, and assess the overshoot against the 15 percent safety margin. For E5, compare the measured delay against the configured 6 second window. For E6, the lower detection limit is almost certainly set by the minimum starting flow of the turbine rather than by the algorithm, and the report should say so explicitly. The E4 result is best presented as a level against time plot with the pump state overlaid and both thresholds marked.

8.4 Latency and behaviour during an outage

E7, and why the headline number is the round trip. The group deliberately reports the command round trip as the primary latency result, not the one way telemetry delay, and the reason is

methodological rather than presentational.

A one way delay is computed as $t_{\text{recv}} - t_{\text{send}}$, where the two terms are read from two different clocks: the server clock and the microcontroller clock. The difference therefore carries the offset between those clocks as a systematic error. The device disciplines its clock over the network time protocol, but the accuracy achievable that way on a microcontroller over Wi-Fi is on the order of tens of milliseconds, which is the same order as the quantity being measured. Immediately after a reboot the group observed an apparent offset of about 1100 milliseconds while the round trip at that same moment was only 187 milliseconds, which is a direct demonstration that the one way figure was reporting clock skew rather than transport delay.

The round trip has no such term. The server stamps the moment it publishes the command and the moment the acknowledgement returns, so both readings come from a single clock and the offset cancels exactly. This is the standard argument behind the round trip time in the network time protocol itself, and it is the reason the group treats the value in Table 8.4.1 as the result of record.

A measurement defect found and corrected. The first implementation carried a device timestamp at one second resolution. The quantisation error of that field alone is up to 500 milliseconds, which exceeds the transport delay being measured, so the computed one way latency was dominated by rounding. The measured median of 844.6 milliseconds over 372 samples was an artefact of the field width, not a property of the network. The group added a parallel field at millisecond resolution while keeping the original field for compatibility, and the same measurement then returned a median of 245.0 milliseconds. Reporting the earlier figure would have overstated the true delay by roughly a factor of three.

The lesson generalises beyond this project: the resolution of an instrument must be finer than the effect it is meant to resolve, otherwise the instrument measures itself.

The outage procedure. The outage test demonstrates the mandatory requirement of the assignment. The procedure isolates the device alone, leaving the broker and the backend connected to each other, because that is the failure the requirement describes. Blocking traffic from the device address for a fixed interval reproduces a lost access point without disturbing the rest of the stack.

The group notes that an earlier attempt stopped the broker process instead. That method is not equivalent: it disconnects the device and the backend simultaneously, and because the telemetry topic uses quality of service zero without retention, the replayed batch is published in the interval between the device reconnecting and the backend resubscribing, and the broker discards it with no subscriber present. The broker log showed the device reconnecting one second before the backend, and none of the sixty replayed messages reached storage. This is recorded here because it is a genuine property of the design rather than an experimental accident: an outage that also restarts the broker loses the buffered history, and the limitations section below revisits it.

Table 8.4.1: Latency, outage behaviour and control policy comparison results.

Quantity	Result
E7: command round trip, median and 95th percentile (ms) (result of record)	225.8 and 575.3, $n = 14$, range 54.5 to 703.2
E7: telemetry one way at millisecond resolution (ms)	245.0 and 708.5, $n = 55$, for comparison only
E7: telemetry one way at the original second resolution (ms)	844.6 and 1482.6, $n = 372$, quantisation artefact
E7: did the pump start on its own during the outage	no, the interlock held throughout
E7: messages buffered and successfully replayed	32, sequence 598 to 629, against a 240 entry buffer
E7: messages lost outright and reconnection time (s)	10 lost, sequence 588 to 597; outage 60.0 s
E8: switching count of configuration A, single threshold	472 per hour, simulated
E8: switching count of configuration B, hysteresis band	8 per hour, simulated
E8: reduction η (%)	98.3, simulated
E8: time outside the target band for A and for B (s)	0.0 and 0.0, simulated

Reading the outage result honestly. Thirty two entries against a buffer of 240 means the buffer was filled to roughly thirteen percent, so a four minute outage remains within capacity and only a longer one would begin to discard the oldest samples.

The ten lost messages deserve more attention than the thirty two that survived. They were not dropped by the network. They were published by the firmware into a socket that the client still believed was open. A transmission control protocol connection severed by a silent packet drop is not detected at the moment of the drop; it is detected when the keep alive interval expires, which is fifteen seconds in this configuration. The firmware buffers only once its client reports a disconnection, so every sample generated inside the detection window is written to a dead socket and lost. The buffer is therefore not a complete guarantee against loss, and the residual gap is bounded by the keep alive interval rather than by the buffer size. Shortening the keep alive would narrow the window at the cost of more frequent traffic.

E8. This experiment demonstrates that the control design is meaningful rather than merely functional. Configuration A uses thresholds of 49 and 51 percent with no timing constraints, approximating a single threshold. Configuration B uses 30 and 70 percent together with both timing constraints. Each configuration runs the same ten minute continuous draw scenario three times, and the reduction is $\eta = (N_A - N_B)/N_A$.

Using one fixed target band from 25 to 85 percent for both configurations is the methodological crux. If the target band were taken to be each configuration own hysteresis band, the wider band would always win for a meaningless reason. Switching counts are obtained from the pump event table by query. Before running on hardware, the group swept ten configurations on the simulator of Section 5.4, because each parameter change on hardware costs roughly fifteen minutes while the model runs in seconds. The values in Table 8.4.1 are the simulated sweep; the hardware repetition is pending the tank assembly.

8.5 Reducing transport latency at the radio

E9. While reading the E7 figures the group noticed that the spread of the round trip was wider than a local wireless network should produce, with a 95th percentile near 900 milliseconds on a link whose signal strength was better than -20 dBm. The cause was traced to the radio rather than to the software: the device firmware never disabled the Wi-Fi power saving mode, and the Arduino port for this microcontroller enables modem sleep by default. In that mode the receiver is powered down between beacon intervals, so a packet arriving at the access point waits until the next scheduled wake before it is processed.

The intervention is a single call placed immediately after the network association. The experiment holds everything else fixed, sends the same number of commands by the same route, and compares the distributions.

Table 8.5.1: Command round trip before and after disabling Wi-Fi power saving. Same device, same access point, same procedure.

Statistic	Power saving on	Power saving off	Change
Minimum (ms)	144	29	-80%
Median (ms)	462	187	-60%
95th percentile (ms)	884	315	-64%
Maximum (ms)	906	402	-56%
Samples	12	12	

The improvement is largest in the tail, which is the expected signature of a scheduling effect rather than a bandwidth effect: the beacon interval sets an upper bound on how long a packet can wait, so removing it truncates the slow cases without changing the fast ones by much.

The cost. The change is not free. Keeping the receiver awake raises the continuous current draw of the module by roughly a factor of two, which is immaterial for a mains powered tank controller but would be decisive for a battery powered node. The group reports the trade off rather than the improvement alone, because a latency figure quoted without its power cost is not an engineering result.

What this does not change. Disabling modem sleep shortens the transport delay. It does not alter the control period of 200 milliseconds or the telemetry period of one second, and it therefore does not change how quickly the tank reacts to a change in water level. The control period is set by the sampling

argument of Section 2.4 and reducing it further would consume power without improving control quality.

8.6 Limitations

The group believes that pointing precisely at where the design falls short carries as much engineering value as presenting what worked.

On hardware, the most serious limitation is that the flow sensor operating point sits near the lower limit of its measuring span, which propagates into the accuracy of the volume computation, the dry run threshold and the lower bound of leak detection; the fix is a sensor whose range matches the present pump. Next, the current measurement is marginal rather than merely coarse, and the arithmetic is worth stating because it decides whether the rule works at all. The fitted pump draws at most 200 milliamperes. A Hall sensor of the 5 ampere variant produces 185 millivolts per ampere, so the full signal is 37 millivolts, and the two to one divider that centres the quiescent point inside the converter range halves it again to 18.5 millivolts. The threshold that declares the pump energised sits at 15 millivolts, which leaves a margin of under a quarter. The 20 and 30 ampere variants of the same part yield 10.0 and 6.6 millivolts respectively, both below the threshold, so with either of those the rule could never fire however correctly the circuit was wired. The load is simply too small for the part: it uses four percent of the sensor span. A shunt resistor of one ohm on the low side would develop 200 millivolts for the same current, an order of magnitude more signal, at the cost of four percent of the supply voltage. Finally, the beam geometry of the ultrasonic sensor is a poor match for the vessel. The transducer sits 15.5 centimetres above the tank floor and radiates a cone of roughly 15 degrees, which has already spread to more than 8 centimetres across by the time it reaches the water. The tank base measures 10 by 10 centimetres, so the edge of the beam strikes the wall on every ping. A stilling tube mitigates this, but the underlying mismatch would be better solved by a vessel wider than the beam or by a measurement principle that does not depend on one.

On measurement, the group no longer relies on a one way difference of two clocks for its headline latency figure, and Section 8.4 explains why; the residual limitation is that the clock synchronisation error was characterised only by observation, never measured against a reference, so the one way column is reported for comparison and not as a result. Two further limitations surfaced during the outage experiment. The buffer protects only the samples generated after the client detects the broken connection, so a window bounded by the fifteen second keep alive interval is lost at the start of every outage regardless of buffer size. And because the telemetry topic uses quality of service zero with a clean session on the subscriber, an outage that also restarts the broker discards the entire replayed batch, since the device reconnects and flushes before the backend has resubscribed; a persistent session with quality of service one on that topic would close the gap at the cost of broker side storage. Repeat counts of three to ten suffice for a mean and a standard deviation but not for rigorous hypothesis testing. The baseline algorithm needs several days of data to converge, so only its startup phase could be evaluated here. On architecture, the system has a single node so distributed systems concerns never arise, the choice of polling over a persistent channel is right at the present scale but will not remain so, and there is no

remote firmware update mechanism.

On the interface, a defect that survived three separate inspections is worth recording because of how it hid rather than because of its size. The query that feeds the history charts applied its row limit after ordering the window in ascending time, so it returned the oldest rows of the window rather than the newest. At the specified rate of one message per second a ten minute window holds six hundred rows and never reaches the two thousand row limit, so the fault was invisible in every ordinary session. Running the simulator at ten times real time raises the rate to roughly five messages per second, the window then holds close to three thousand rows, and the charts silently froze in the past while the live status panel beside them continued to update. Nothing failed and nothing was logged; the two halves of the same screen simply disagreed.

The general lesson is that a pagination limit at the query layer must truncate from the end that is being discarded, not from the end being displayed. Ordering ascending and then limiting is correct only while the result set is known to fit, which makes it a latent fault that waits for the data rate to rise. The corrected query selects the newest rows first and reorders them for plotting. The group notes that the defect was found only because the acceleration factor of the simulator pushed the system past a threshold the specified rate never approaches, which is an argument for exercising a system deliberately outside its nominal operating point rather than only at it.

Future directions include model based diagnosis by running the tank model in parallel with the real device [14], sensor fusion with a Kalman filter, continuous speed control through pulse width modulation instead of on off switching, multi node expansion, and migration to a dedicated time series store. The group considers the first of these the most valuable academically, since the hydraulic model already exists for the parameter sweep, so turning it into a parallel observer is a natural extension that needs no additional hardware.

8.7 Faults found on the bench

The faults recorded here were all found after the prototype was assembled and nominally working. The group presents them because each was invisible from the code alone and each was resolved by a measurement rather than by argument.

8.7.1 An inverted relay made the firmware believe the pump was off

The most serious fault was also the simplest. The configuration declared the relay module active low, so the firmware drove the control pin high to stop the pump. The module fitted was active high. Every command to stop therefore started the pump, and the telemetry reported `pump: false` throughout. The tank filled from 0.2 to 11.2 centimetres and came close to overflowing while the interface displayed a stopped pump and an overflow lock.

The fault also corrupted the current measurement, in a way that concealed it. The quiescent offset of the Hall sensor is tracked by an exponential average that updates only while the firmware believes the

pump is off. Because the firmware believed that permanently, the average absorbed the running current of the pump as its own zero, and the reported current settled near zero while the motor ran.

Determining the true polarity required care, because the obvious method gives the wrong answer. The Hall current sensor is bidirectional: its output rests at mid scale and moves up or down depending on the direction of current through the primary conductor. A larger reading therefore does not imply a larger current, and the first version of the group's test concluded the opposite of the truth on exactly that basis. The quantity that does separate the two states is the *ripple*, because a brushed motor draws a visibly rough current through its commutator while an open relay draws a flat one. Measured over two rounds, the control pin low gave a mean of 1545.88 millivolts with 4.47 millivolts of ripple, and the pin high gave 1516.54 millivolts with 27.05 millivolts of ripple. The ripple ratio of 6.0 identifies the energised state unambiguously and does not depend on which way the load is wired, whereas the means differ by only 29 millivolts and in the misleading direction.

8.7.2 An automatic direction level shifter consumed a third of the echoes

The echo line of the ultrasonic sensor was initially routed through a TXS0108E translator. That part is intended for bidirectional open drain buses: it carries integrated pull up resistors of about 10 kilohms on both ports and a one shot circuit that accelerates edges. The echo output of an HC-SR04 is a strong push pull driver, and a pulse width is the only carrier of information in the signal, so both mechanisms work against it. Replacing the translator with a two resistor divider on the echo line alone changed the miss rate from between 28 and 30 percent of pings to zero in 179 consecutive samples.

8.7.3 Fifty hertz on a floating signal line reads as plausible flow

Both flow inputs reported between 0.26 and 0.51 litres per minute with the pump stopped. Converting the reading back to its underlying pulse rate showed exactly 50.00 hertz on one channel, unchanging across every sample. The control experiment was to close the drain valve, after which no flow of any kind was physically possible; the channel continued to report 50.00 hertz. A signal locked to the mains frequency and indifferent to whether water can flow is induced interference, not measurement.

The cause is impedance. The internal pull up of the microcontroller is about 45 kilohms, which leaves the line high enough in impedance to act as an antenna. The specified remedy, an external 4.7 kilohm pull up to the 3.3 volt rail, lowers the source impedance by an order of magnitude. A second and larger disturbance appears only while the pump runs, when both channels rise together to about 4750 hertz, a figure that would correspond to 48 litres per minute against a measured pump delivery of 0.36. Two independent sensors agreeing to within 0.7 percent are not measuring water; they are sharing a common mode disturbance injected by the motor.

8.7.4 A silently truncated message stopped the database without stopping anything else

Adding two validity fields to the telemetry payload pushed its length past the 448 byte buffer that served it. The JSON library truncates without raising an error and still returns a string, so the device published well formed nonsense. On the server the parse exception was raised inside a callback whose exceptions the client library discards. The result was a system in which the device published once per second, the broker delivered every message, the health endpoint reported the service as connected, and the database received nothing at all for several minutes while the dashboard displayed a frozen value.

The lesson the group takes is that a serialiser which truncates silently and a callback which swallows exceptions are individually defensible and jointly produce a failure with no symptom at the point of failure. Both ends now report the condition explicitly, and the buffer carries margin above the maximum payload rather than matching it.

8.7.5 A command that worked but was never confirmed

Manual pump commands were executed by the device, yet the command log kept them as pending indefinitely, while mode commands were confirmed normally. The broker log showed the difference: acknowledgements of mode commands were 118 bytes long, acknowledgements of pump commands exactly 256 bytes, the size of the acknowledgement buffer.

The cause was an interaction between two libraries. The MQTT client delivers an incoming message as a pointer into its internal buffer, and the same buffer is reused for outgoing messages. The JSON parser, handed a non-constant pointer, parses in place and returns string references into that buffer rather than copies. A pump command starts the pump, which publishes the new pump state before the acknowledgement is sent; that publish overwrote the buffer, so the command identifier being echoed back had already become arbitrary bytes. Mode commands publish nothing in between and were unaffected. The fix copies the incoming message into a buffer owned by the handler before parsing. The general point is that a zero-copy parser makes the lifetime of the input part of the program's correctness, and that lifetime is decided by another library.

8.7.6 A robust filter gated by a fragile statistic

After the level filter was widened to a fifteen sample median, the group counted, for every ping, which validation stage rejected it. In one ordinary fill of 605 pings, 244 were rejected, and 98 percent of those rejections came from a single stage: the check that the spread of the window stayed below a limit. Missing echoes accounted for only 2 percent. The sensor was answering almost every time.

The spread had been measured as maximum minus minimum. That statistic is controlled entirely by the single most extreme sample, so one stray echo from the tank floor among fifteen good ones was enough to discard the whole window. The median was chosen precisely because it tolerates isolated outliers; gating it with the statistic most sensitive to isolated outliers undid that choice. Replacing the range with the interquartile range, which ignores the top and bottom quarter of the window, raised the

acceptance rate from 60 to 93 percent in otherwise identical conditions. The interquartile range still grows when the sensor genuinely alternates between the water surface and the floor, because then roughly half the samples sit in each group, which is exactly the case the check exists to reject.

8.7.7 The level sensor is excellent, and unusable, in the same installation

With the water still, twenty consecutive samples read between 3.87 and 3.89 centimetres, a standard deviation of 0.010 centimetres. With the pump running, the same sensor collapses to reporting an empty tank while the tank is demonstrably filling. The group verified that the validity gates were not the cause by disabling them: the raw reading itself falls to the empty value, so the gates were reporting the data correctly rather than rejecting it wrongly.

This pair of results is the clearest argument in the project for layered protection. Throughout the interval in which the level reading was false and the validity flag had been forced true, the rule that stops the pump when the level fails to rise still fired and still stopped it. That rule consults the clock and the level history, never the validity flag, which is precisely why it survived.

8.8 Conclusion

The project has delivered a complete closed loop water management system spanning all five architectural layers from physical sensing to a real time monitoring interface, with every line of source code written by the group.

The main technical contribution rests on three points. First, the architectural decision to place the entire control law and all diagnostic logic inside the edge microcontroller, derived from the rule that any operation requiring an immediate safe stop must be computed at the edge. That decision turns the outage requirement from a feature to be added into a natural consequence of the design. Second, the treatment of safety constraints: instead of scattering checks across the code, the group funnels every pump starting path through one guard function, so the requirement that manual mode cannot override safety is guaranteed by program structure rather than by the diligence of whoever writes the code; alongside this, four independent overflow protection layers exist, of which the last two remain effective even when the sensor has failed or the microcontroller has lost power. Third, the treatment of data: both main sensors were calibrated against physical references instead of trusting datasheet figures, the control parameters were selected from a sweep on the hydraulic model, and the value of the control policy was demonstrated by a direct comparison experiment on a common scenario under a common quality criterion.

In terms of learning outcomes, building this system tied together material that the first six weeks of the course present separately, from sensor physics and analog to digital conversion, through power stage design, to application layer messaging and time series data infrastructure. Making those pieces work together on a real prototype was the part from which the group learned the most.

8.9 Member contributions

Table 8.9.1: Distribution of work among group members.

Member	Area of responsibility
[Full name of member 1], [ID]	[Fill in area of responsibility]
[Full name of member 2], [ID]	[Fill in area of responsibility]
[Full name of member 3], [ID]	[Fill in area of responsibility]
[Full name of member 4], [ID]	[Fill in area of responsibility]
[Full name of member 5], [ID]	[Fill in area of responsibility]

Suggested grouping for the five members: hardware and circuitry; firmware and control; calibration and experiments; backend and database; interface and report. The assignment requires every member to present and answer technical questions, so the table records who led each area rather than limiting anyone scope of understanding.

The schedule spans six weeks: week one freezes the protocol specification, sets up the repository, runs the parameter sweep and orders components; week two assembles the circuit in seven steps and reaches the broker; week three runs E1 and E2, loads the calibration coefficients and completes the backend and interface; week four implements and tunes the fault detection rules; week five runs E3 through E8, collects the data and records a backup video; week six writes the report, prepares the slides and rehearses. Deliverables are a working physical prototype, the firmware with four unit test programs, the backend and dashboard source, the simulator, the protocol specification document, the raw data set and the backup demonstration video.

Bibliography

- [1] H. T. Thien, “IoT foundations and hardware architectures,” Lecture notes, IoT Foundations and Applications, Week 1, Ho Chi Minh City University of Technology and Education, 2026.
- [2] ITU-T, “Recommendation Y.2060: Overview of the Internet of Things,” International Telecommunication Union, 2012.
- [3] C. E. Shannon, “Communication in the presence of noise,” *Proceedings of the IRE*, vol. 37, no. 1, 1949, pages 10 to 21.
- [4] Espressif Systems, “ESP32 series datasheet,” Technical documentation, 2024. [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>
- [5] H. T. Thien, “IoT application layer messaging,” Lecture notes, IoT Foundations and Applications, Week 3, Ho Chi Minh City University of Technology and Education, 2026.
- [6] OASIS, “MQTT version 3.1.1,” OASIS Standard, 2014, published 29 October 2014.
- [7] E. Rescorla, “The transport layer security (TLS) protocol version 1.3,” RFC 8446, IETF, 2018.
- [8] H. T. Thien, “IoT cloud platforms, big data and edge computing,” Lecture notes, IoT Foundations and Applications, Week 4, Ho Chi Minh City University of Technology and Education, 2026.
- [9] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [10] H. T. Thien, “RESTful web API design and JSON data serialization,” Lecture notes, IoT Foundations and Applications, Week 5, Ho Chi Minh City University of Technology and Education, 2026.
- [11] B. Blanchon, “ArduinoJson: JSON library for embedded C++,” 2024. [Online]. Available: <https://arduinojson.org/>
- [12] K. Ogata, *Modern Control Engineering*, 5th ed. Prentice Hall, 2010.
- [13] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008.
- [14] R. Isermann, *Fault-Diagnosis Systems: An Introduction from Fault Detection to Fault Tolerance*. Springer, 2006.
- [15] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1/2, 1954, pages 100 to 115.
- [16] F. M. White, *Fluid Mechanics*, 7th ed. McGraw-Hill, 2011.
- [17] H. T. Thien, “Connectivity and network protocols,” Lecture notes, IoT Foundations and Applications, Week 2, Ho Chi Minh City University of Technology and Education, 2026.

- [18] H. T. Thien, “Frontend development, mobile apps and edge AI,” Lecture notes, IoT Foundations and Applications, Week 6, Ho Chi Minh City University of Technology and Education, 2026.
- [19] SQLite Consortium, “SQLite documentation,” 2024. [Online]. Available: <https://www.sqlite.org/docs.html>
- [20] I. Fette and A. Melnikov, “The WebSocket protocol,” RFC 6455, IETF, 2011.
- [21] Eclipse Foundation, “Eclipse Mosquitto: An open source MQTT broker,” 2024. [Online]. Available: <https://mosquitto.org/>
- [22] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.
- [23] J. P. Bentley, *Principles of Measurement Systems*, 4th ed. Pearson Prentice Hall, 2005.
- [24] JCGM, “Evaluation of measurement data: Guide to the expression of uncertainty in measurement,” JCGM 100:2008, 2008.